Lab 2 – Nhận diện cảm xúc khuôn mặt bằng Deep Learning

Sinh viên: **Hồ Vĩnh Hưng** · MSSV: **CE202073** · Giảng viên: TS. Nguyễn Đình Vinh · FPT University Cần Thơ

Tóm tắt. Hệ thống nhận diện 7 cảm xúc từ luồng camera điện thoại (IP camera qua Wi-Fi), chạy trên CPU laptop. Huấn luyện trên FER-2013 và đánh giá trên PrivateTest (3.589 ảnh): **M1** (CNN tự thiết kế, 4,82M tham số) đạt **70,86%** accuracy, macro-F1 **0,695**; **M2** (ResNet18 transfer learning, 11,18M tham số) đạt **71,30%**, macro-F1 **0,704**. Cả hai vượt ngưỡng của đề (60% và 65%). Trên CPU, thời gian suy luận của cả hai khoảng 5 ms/khuôn mặt khi chạy bằng ONNX Runtime.

1. Dữ liệu và tiền xử lý

Bộ dữ liệu: chỉ dùng FER-2013 (ảnh xám 48×48, 7 lớp), không dùng thêm dữ liệu ngoài. Chia tập theo giao thức chuẩn của các bài báo:

- **Train** = *Training*: 28.709 ảnh. Loại 12 ảnh trơn (độ lệch chuẩn pixel < 2, lỗi thu thập), còn 28.697 ảnh.
- **Validation** = *PublicTest*: 3.589 ảnh, dùng để chọn checkpoint.
- **Test** = *PrivateTest*: 3.589 ảnh, chỉ dùng **một lần** ở cuối.

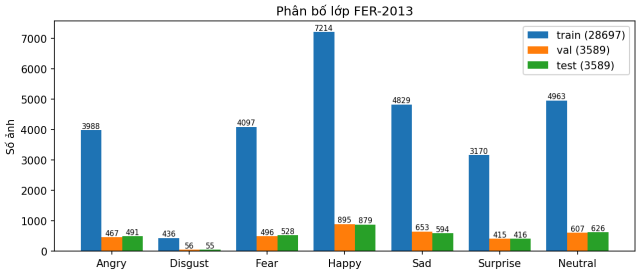
Lớp	Train	Tỉ lệ	Val	Test	Weight (nghịch đảo)	Weight dùng (bậc 0,5)
Angry	3.995	13,9%	467	491	1,03	**1,06**
Disgust	436	1,5%	56	55	9,41	**3,20**
Fear	4.097	14,3%	496	528	1,00	**1,04**
Happy	7.215	25,1%	895	879	0,57	**0,79**
Sad	4.830	16,8%	653	594	0,85	**0,96**
Surprise	3.171	11,0%	415	416	1,29	**1,19**
Neutral	4.965	17,3%	607	626	0,83	**0,95**

Mất cân bằng: Happy nhiều gấp khoảng 16,5 lần Disgust. Biện pháp xử lý gồm:

- **Class weight bậc 0,5**, tức $w_c \propto (N/n_c)^{0,5}$, chuẩn hoá để trung bình theo mẫu bằng 1. Nghịch đảo đầy đủ cho Disgust trọng số gấp 16 lần Happy, khiến mô hình đoán Disgust tràn lan. Bậc 0,5 chỉ gấp khoảng 4 lần.
- **Không oversampling**, vì nhân 436 ảnh Disgust lên hàng nghìn ảnh dễ khiến mô hình học thuộc.
- **Label smoothing 0,1** để giảm ảnh hưởng của nhãn nhiễu.
- Chọn checkpoint theo **val macro-F1** thay vì accuracy.

Tiền xử lý và augmentation:

- Ảnh được chia 255. M1 chuẩn hoá theo mean/std của tập train; M2 chuẩn hoá theo mean/std của ImageNet.
- Augmentation chạy **trên GPU theo batch**: lật ngang (p = 0,5), xoay ±10°, zoom ±10%, dịch ±8%, độ sáng ±0,2, tương phản ×(1 ± 0,2), random erasing (p = 0,25, 2–12% diện tích). Các phép này mô phỏng điều kiện camera điện thoại (thiếu sáng, che khuất, crop lệch).



Hình 1. Phân bố lớp



Hình 2. Ví dụ augmentation (cột đầu là ảnh gốc)

2. Kiến trúc mô hình

Thiết kế chung – tiền xử lý nằm trong mô hình. Mọi mô hình đều nhận cùng một loại đầu vào: tensor 1×48×48 trong khoảng [0, 1]. Các bước riêng của từng mô hình (upscale, nhân 3 kênh, chuẩn hoá mean/std) nằm trong lớp `InputAdapter` ở đầu mô hình và được export luôn vào ONNX. Nhờ vậy, notebook, app realtime và ONNX dùng **chung một hàm tiền xử lý**, không thể lệch nhau giữa lúc train và lúc chạy.

M1 – CNN tự thiết kế (4,82M tham số)

```
Input 1×48×48 → chuẩn hoá (mean/std train)
Block ×4: [Conv3×3–BN–ReLU]×2 → MaxPool2 →
Dropout2d
  kênh 64 → 128 → 256 → 512 ; dropout 0,1 →
0,4
  kích thước 48 → 24 → 12 → 6 → 3
GAP → FC256–BN–ReLU → Dropout 0,5 → FC7
```

- Hai conv 3×3 có receptive field bằng một conv 5×5 nhưng ít tham số hơn và thêm một lớp phi tuyến.
- Conv không có bias vì BatchNorm đứng ngay sau đã có tham số dịch β.
- Dropout2d bỏ cả feature map, phù hợp với conv hơn Dropout thường. Tỉ lệ dropout tăng dần theo độ sâu.
- Global Average Pooling thay cho Flatten giúp bớt khoảng 1M tham số ở lớp FC.

M2 – Transfer learning ResNet18 (11,18M)

```
Input 1×48×48 → bilinear lên 112×112 → nhân 3
kênh
→ chuẩn hoá ImageNet → ResNet18 (pretrained
ImageNet)
→ GAP 512 → Dropout 0,3 → FC7
```

- **Input 112** thay vì 224: ảnh gốc chỉ 48 px nên upscale lên 224 không thêm thông tin mà tốn tính toán gấp 4 lần. Còn ở kích thước 48, feature map cuối chỉ 2×2, quá thô. Ở 112, feature map cuối là 4×4.
- Chọn ResNet18 vì cân bằng tốt giữa accuracy và tốc độ CPU. ResNet50 chậm hơn khoảng 2–3 lần mà chỉ hơn khoảng 1%.
- Fine-tune 2 giai đoạn: 2 epoch đầu đóng băng backbone, chỉ train head; sau đó mở khoá toàn bộ với discriminative LR.

3. Siêu tham số

Tham số	M1 (CNN)	M2 (ResNet18)	Lý do
Optimizer	AdamW	AdamW	Weight decay tách rời đúng nghĩa L2. Không áp decay cho bias và tham số BN.
LR / scheduler	3e-3, OneCycle, warm-up 15%	head 1e-3, backbone 3e-4, OneCycle, warm-up 10%	Warm-up giúp BN và Adam ổn định lúc đầu. Backbone đã pretrained nên chỉ cần chỉnh nhẹ.
Weight decay	0,05	0,01	M1 train từ đầu nên dễ overfit. Ở M2, decay mạnh làm mất đặc trưng ImageNet.
Epoch / batch	60 / 128	30 (2 epoch đóng băng) / 128	Early stopping theo val macro-F1 (patience 15 / 10).
Loss	CE + class weight 0,5 + smoothing 0,1	giống M1	Xử lý mất cân bằng và nhãn nhiễu.
Khác	AMP fp16, seed 42	AMP fp16, seed 42	Nhanh hơn khoảng 1,5–2 lần trên RTX 4060 Laptop.

4. Kết quả và so sánh M1 – M2 (M3)

Mô hình	Accuracy	Macro-F1	Tham số	ONNX	CPU PyTorch	CPU ONNX	FPS model-only	FPS end-to-end	Train
M1 CNN tự thiết kế	**70,86%**	0,695	4,82M	18,4 MB	8,20 ms	**4,97 ms**	**201**	[__]	12,5 phút
M2 ResNet18 (TL)	**71,30%**	**0,704**	11,18M	42,6 MB	14,95 ms	5,17 ms	193	≈30–40*	**7,2 phút**

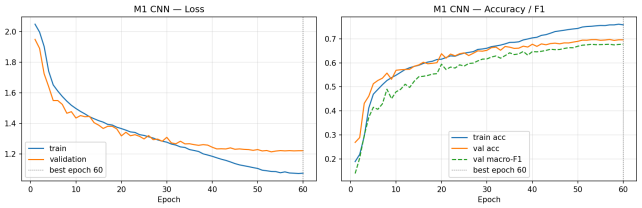
Tốc độ model-only: 1 khuôn mặt, CPU, trung vị của 300 lần chạy sau warm-up. FPS end-to-end: đo khi chạy `app.py` với camera điện thoại (IP Webcam, 640×480, laptop cắm sạc), bao gồm cả đọc camera, phát hiện mặt, mô hình và vẽ. M1: trung bình của 2.188 khung hình liên tục. *M2: giá trị hiển thị trên khung hình trong lúc demo. Cả hai đều vượt xa ngưỡng 10 FPS và bị giới hạn bởi tốc độ phát của camera điện thoại (khoảng 30 khung/giây), không phải bởi mô hình.

F1 theo lớp	Angry	Disgust	Fear	Happy	Sad	Surprise	Neutral

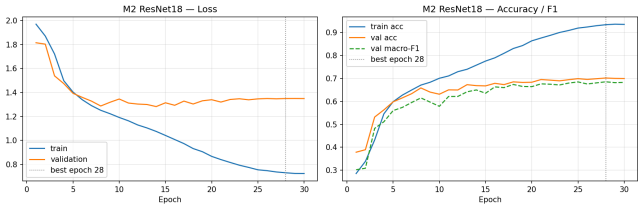
M1	0,642	0,732	0,511	**0,908**	**0,574**	0,803	**0,693**
M2	**0,659**	**0,752**	**0,546**	0,903	0,556	**0,832**	0,684

- **Độ chính xác gần như ngang nhau.** M2 chỉ hơn 0,44 điểm accuracy, tương đương khoảng 16 ảnh. Mức chênh này nhỏ hơn sai số chuẩn của mỗi phép đo (khoảng ±0,76% với 3.589 ảnh). Nguyên nhân là ảnh 48×48 ít chi tiết, nên lợi thế của đặc trưng ImageNet bị hạn chế. Thêm nữa, M1 được regularize mạnh (dropout tăng dần, augmentation, weight decay).
- **Tốc độ:** M2 có số tham số gấp 2,3 lần. Khi chạy PyTorch eager, M2 chậm hơn M1 khoảng 1,8 lần, nhưng khi chạy bằng ONNX Runtime thì chỉ chậm hơn khoảng 4%. Cả hai đều rất xa ngưỡng 10 FPS, nên nút thắt nằm ở camera và detector, không phải ở bộ phân loại.
- **Ưu điểm của M2:** hội tụ nhanh hơn (7,2 so với 12,5 phút) và tốt hơn ở Fear, Surprise, Angry, Disgust. **Ưu điểm của M1:** nhẹ gấp 2,3 lần và tốt hơn ở Sad, Neutral.

Đồ thị huấn luyện (M5)



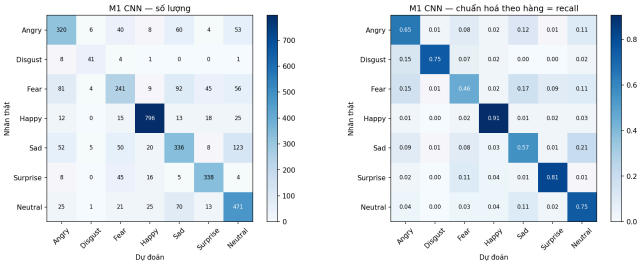
Hình 3. Loss và accuracy train/val của M1



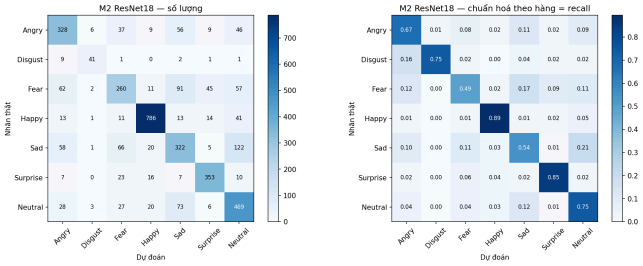
Hình 4. Loss và accuracy train/val của M2

- **M1:** chạy đủ 60 epoch, epoch tốt nhất là **epoch 60** (val acc 69,6%, val F1 0,678). Train acc ở cuối chỉ 75,8%, gần val acc, nghĩa là M1 **gần như không overfit** nhờ dropout, augmentation và weight decay. Val F1 vẫn còn tăng ở epoch cuối, nên train thêm (khoảng 80–100 epoch) có thể cải thiện thêm.
- **M2:** epoch tốt nhất là **epoch 28** (val acc 70,2%, val F1 0,685). Train acc lên tới 93,4% trong khi val acc chỉ khoảng 70%, tức là **overfit rõ**. Val loss thấp nhất ở epoch 14 (1,28) rồi đi ngang ở mức khoảng 1,35, trong khi val F1 vẫn tăng nhẹ: mô hình đoán đúng thêm một số ảnh nhưng trở nên quá tự tin ở các ảnh sai, làm loss tăng. Vì vậy checkpoint được chọn theo F1 chứ không theo loss.
- Ở các epoch đầu, train acc thấp hơn val acc vì được tính trên ảnh đã augmentation và có dropout. Đây là hành vi bình thường.

5. Phân tích lỗi (M4)



Hình 5. Confusion matrix của M1 (bên phải chuẩn hoá theo hàng, tức recall)



Hình 6. Confusion matrix của M2

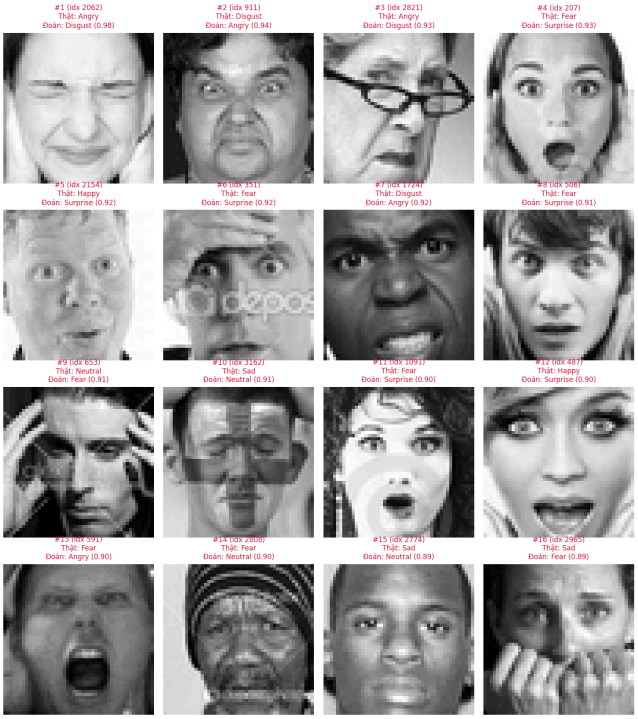
Các cặp nhầm lẫn chính (tỉ lệ trên số ảnh của lớp thật, M1 / M2):

Thật → Đoán	M1	M2	Giải thích
Sad → Neutral	21% (123)	21% (122)	Sad cường độ thấp gần như không có cử động cơ mặt, giống Neutral.
Fear → Sad	17% (92)	17% (91)	Cả hai đều có lông mày nhướng và kéo vào giữa (AU1 + AU4).
Disgust → Angry	15% (8)	16% (9)	Cả hai đều nhíu mày (AU4); Disgust chỉ có 55 ảnh test.
Fear → Angry	15% (81)	12% (62)	Miệng há và căng mặt xuất hiện ở cả hai.

Surprise → Fear	11% (45)	6% (23)	Lông mày nhướng, mắt mở to (AU1, AU2, AU5); khác nhau ở độ căng môi (AU20), chi tiết khó thấy ở 48×48.

Nhận xét:

- **Fear là lớp khó nhất**, recall chỉ 0,46 với M1 và 0,49 với M2. Ảnh Fear bị phân tán sang Sad, Angry, Neutral và Surprise.
- **Happy dễ nhất** với recall khoảng 0,9, vì nụ cười (AU12) là tín hiệu rõ và khác biệt.
- **Hai mô hình nhầm theo mẫu gần như giống hệt nhau.** Điều này cho thấy giới hạn nằm ở dữ liệu (biểu cảm chồng lấn, nhãn nhiễu) chứ không ở kiến trúc. M2 chỉ cải thiện rõ ở Surprise → Fear và Fear → Angry.



Hình 7. 16 ảnh M1 đoán sai với độ tin cậy cao nhất (0,89–0,98)



Hình 8. 16 ảnh M2 đoán sai với độ tin cậy cao nhất (0,94–0,97)

Nhận xét ảnh dự đoán sai

Mô hình đoán sai mà vẫn tự tin trên 0,89 thường là dấu hiệu của **ảnh mơ hồ hoặc nhãn nhiễu**, không phải do mô hình chưa học. Nhận xét đầy đủ cho cả 32 ảnh (16 của M1, 16 của M2) nằm trong notebook. Dưới đây là các nhóm lỗi chính kèm ví dụ (M1#k là ảnh thứ k ở Hình 7, M2#k là ảnh thứ k ở Hình 8):

Nhóm lỗi	Ví dụ	Giải thích
Cặp biểu cảm có chung cơ mặt	M1#4, #8: Fear→Surprise; M1#2, #7: Disgust→Angry; M1#15: Sad→Neutral; M2#12: Surprise→Fear	M1#4 có mắt mở to, lông mày nhướng, miệng há tròn, tức là nguyên mẫu của Surprise. M1#7 vừa nhíu mày (Angry) vừa nhăn mũi, nhe răng (Disgust). M2#12 có môi kéo ngang cộng lông mày nhướng, chính là tổ hợp AU của Fear.
Nhãn nhiễu hoặc mơ hồ	M1#1: Angry→Disgust (0,98); M1#5: Happy→Surprise; M1#9: Neutral→Fear; M1#12; M2#14: Angry→Sad	M1#1 có mũi nhăn và môi trên nâng, đúng đặc trưng Disgust. M1#9 có hai tay ôm đầu mà lại gán nhãn Neutral. M2#14 nhắm nghiền mắt, miệng méo như đang khóc. M1#12 là biểu cảm hỗn hợp vui và ngạc nhiên, một nhãn duy nhất không đủ mô tả.
Che khuất hoặc watermark	M1#6 (watermark depositphotos), M1#16 (tay che miệng), M2#2 (kính), M2#4 (vật che miệng), M2#13 (watermark iStock)	Khi miệng hoặc lông mày bị che, mô hình chỉ dựa vào vùng mắt. Mắt mở to dẫn đến đoán Fear hoặc Surprise.
Tư thế, chất lượng ảnh	M2#1 (mặt nghiêng gần hẳn), M2#6, M2#8 (mặt nhỏ, mờ), M2#9 (mờ do chuyển động), M1#3 (nghiêng, kính)	Mất một nửa khuôn mặt hoặc mất chi tiết. M2 sai nhiều hơn ở nhóm này: đặc trưng ImageNet không giúp nhiều với tư thế đầu.

Nằm ngoài phân phối	M1#10 (ảnh lỗi khảm khối), M1#11 = M2#15 (tranh vẽ), M2#3 (ảnh hoá trang)	Đây là lỗi của dữ liệu FER-2013. Ảnh idx 1091 và idx 2808 nằm trong top lỗi của **cả hai** mô hình, cho thấy lỗi nằm ở ảnh chứ không ở kiến trúc.

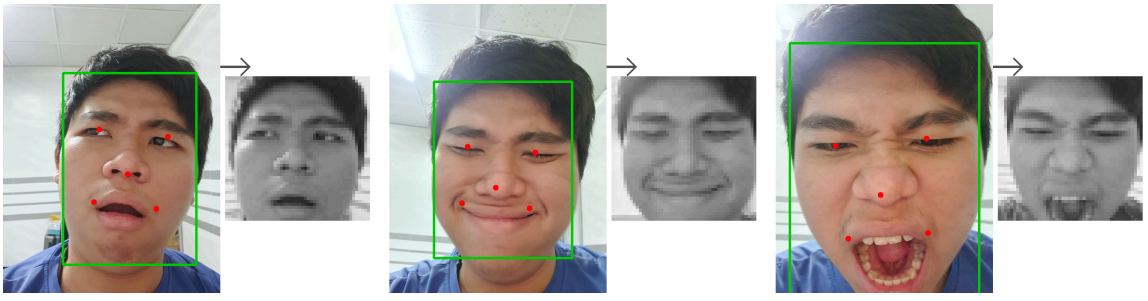
Hướng cải thiện:

1. Dùng nhãn **FER+** (mỗi ảnh được 10 người gán nhãn) để giảm nhãn nhiễu.
2. Thêm dữ liệu mặt nghiêng (RAF-DB).
3. Ensemble M1 + M2 kết hợp TTA lật ngang, vì hai mô hình sai ở một phần ảnh khác nhau.
4. Với M1: train thêm epoch (val F1 vẫn đang tăng). Với M2: tăng augmentation hoặc weight decay để giảm overfit.
5. Ở ứng dụng realtime, crop theo landmark mắt và xoay thẳng khuôn mặt (mục 6).

6. Ứng dụng thời gian thực

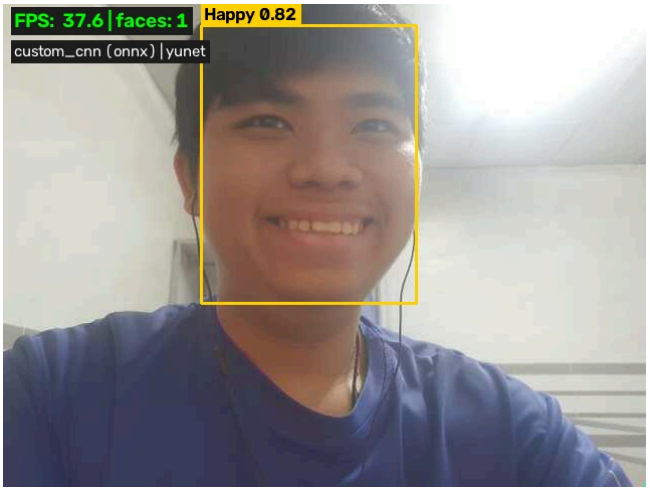
```
Camera điện thoại —Wi-Fi→ Thread đọc camera (chỉ giữ khung mới nhất, tự kết nối lại)
 → YuNet (OpenCV DNN) phát hiện mặt + 5 landmark, chạy mỗi 2 khung, ghép box theo IoU
 → xoay thẳng theo 2 mắt, crop vuông theo vị trí mắt như FER, xám 48×48 (INTER_AREA)
 → M1/M2 chạy ONNX Runtime, gộp mọi khuôn mặt vào một batch → làm mượt EMA theo từng mặt
 → vẽ box + nhãn + confidence (dưới ngưỡng 0,4: thêm "?" và khung xám) + FPS
```

- **Face detector YuNet** thuộc nhóm DNN face detector của OpenCV. Model chỉ khoảng 230 KB, bắt mặt nghiêng và thiếu sáng tốt hơn Haar. Khi mặt chiếm gần hết khung hình (selfie sát camera), app tự thử lại ở độ phân giải thấp hơn.
- **Crop khớp với FER-2013.** Box của detector co giãn theo góc nhìn và tóc, nên app căn crop theo 2 mắt: khoảng cách 2 mắt bằng 43% bề ngang và đường mắt nằm ở 38% chiều cao, giống ảnh FER (Hình 9). Nếu crop theo box thì khoảng cách 2 mắt chỉ còn khoảng 30% bề ngang, ảnh dính cả vai và nền, lệch phân phối so với lúc train.
- **Độ trễ thấp.** Đọc camera bằng thread riêng nên luôn xử lý khung mới nhất, tránh việc buffer dồn làm trễ vài giây.
- **Mất kết nối.** App hiện thông báo "CAMERA DISCONNECTED", tự kết nối lại khi có mạng, không crash hay treo. Đã thử bằng cách tắt Wi-Fi giữa chừng.
- **Nhãn ổn định.** Làm mượt bằng EMA theo từng khuôn mặt, công thức $p_t = 0{,}5 \cdot p_{mới} + 0{,}5 \cdot p_{t-1}$, nên nhãn không nhấp nháy giữa các khung.
- **Tham số dòng lệnh:** `--source` nhận URL camera, ảnh, video hoặc số webcam; `--model cnn|transfer`. Các giá trị URL, đường dẫn model, ngưỡng confidence nằm trong `config.yaml`, không hard-code.

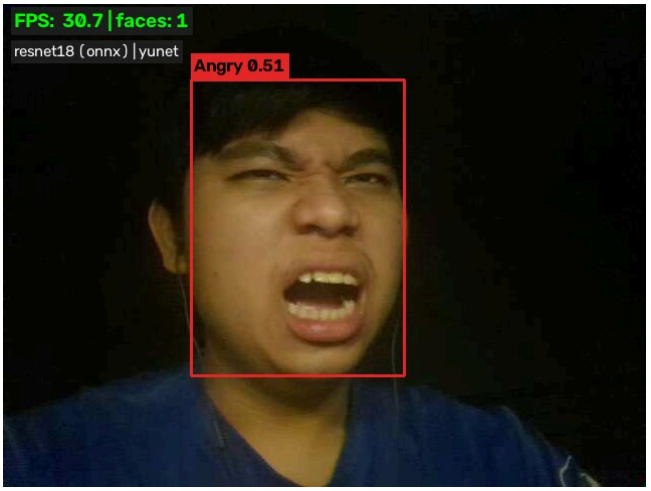


Hình 9. Ảnh chụp từ điện thoại → YuNet (box + 5 landmark) → crop 48×48 đưa vào mô hình. Ảnh đầu tiên có đầu nghiêng đã được xoay thẳng.

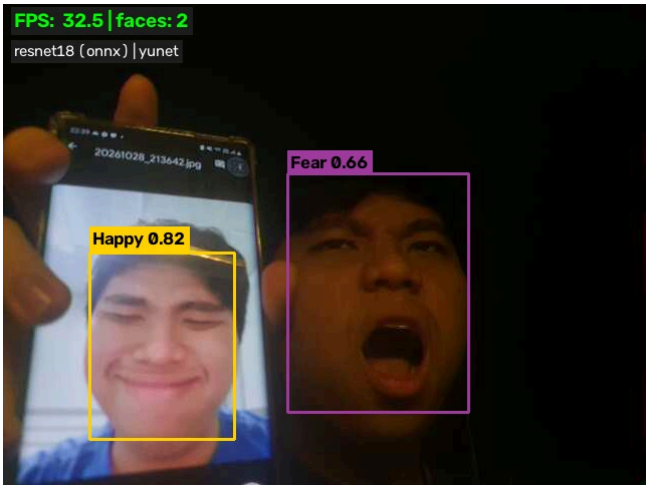
Demo với camera điện thoại



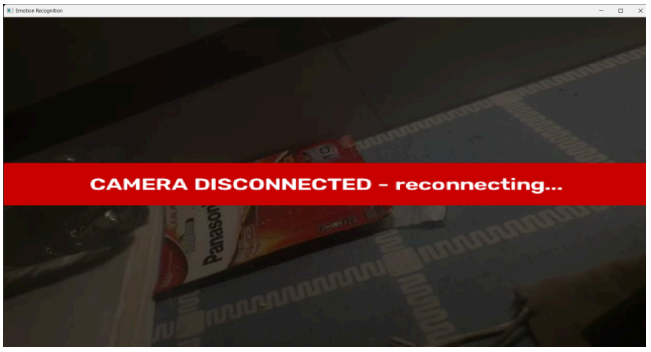
Hình 10. M1 (--model cnn): Happy 0,82, FPS 37,6



Hình 11. M2 (--model transfer): Angry 0,51 khi nhăn mặt, phòng thiếu sáng



Hình 12. M2, 2 khuôn mặt trong cùng khung (mặt thật + ảnh trên điện thoại)



Hình 13. Tắt Wi-Fi điện thoại: app báo mất kết nối, không crash, tự kết nối lại

7. Kết luận

Cả hai mô hình vượt ngưỡng yêu cầu: M1 đạt 70,86% và M2 đạt 71,30% accuracy trên FER-2013 PrivateTest. Mức này gần với kết quả tốt nhất từng công bố cho một mô hình đơn lẻ (khoảng 73%) và cao hơn độ đồng thuận của con người trên bộ dữ liệu này (khoảng 65 ± 5%).

Giới hạn chính nằm ở dữ liệu: nhãn nhiễu, nhiều cặp biểu cảm có chung cơ mặt, và ảnh 48×48 nghèo chi tiết. Vì vậy, thay kiến trúc lớn hơn chỉ mang lại thêm rất ít độ chính xác. Hướng hiệu quả hơn là cải thiện nhãn (FER+), cải thiện dữ liệu, và làm cho tiền xử lý lúc chạy realtime khớp với lúc train.